UNIVERSITY OF CAPE TOWN
Department of Computer Science

Abstract

The document designs a system architecture diagram for a networked chat application. The custom application protocol, sequence of messages, error handling and trade-offs of communicating entities are clearly defined.

Amahle Mbambo | Karabo Nkambule | Tracey Letlape

MBMAMA007@myuct.ac.za | NKMKAR002@myuct.ac.za | LTLTRA001@myuct.ac.za

Contents

1. System Architecture Diagram 2

2. Application-Level Protocol Specification 4

Custom Application Protocol Design 4

Protocol Overview 4

1. Roles of Communicating Entities 4
2. Message Categories 5
3. Message Structure 5
4. The valid sequence of messages: 6
5. Expected Behavior in Normal and Error conditions: 8

3. Design Rationale and Trade-Off Discussion 12

Design Rationale 12

Client-Server Model 14

Peer-To-Peer Model 15

4. Use of AI Declaration. 15

1. System Architecture Diagram

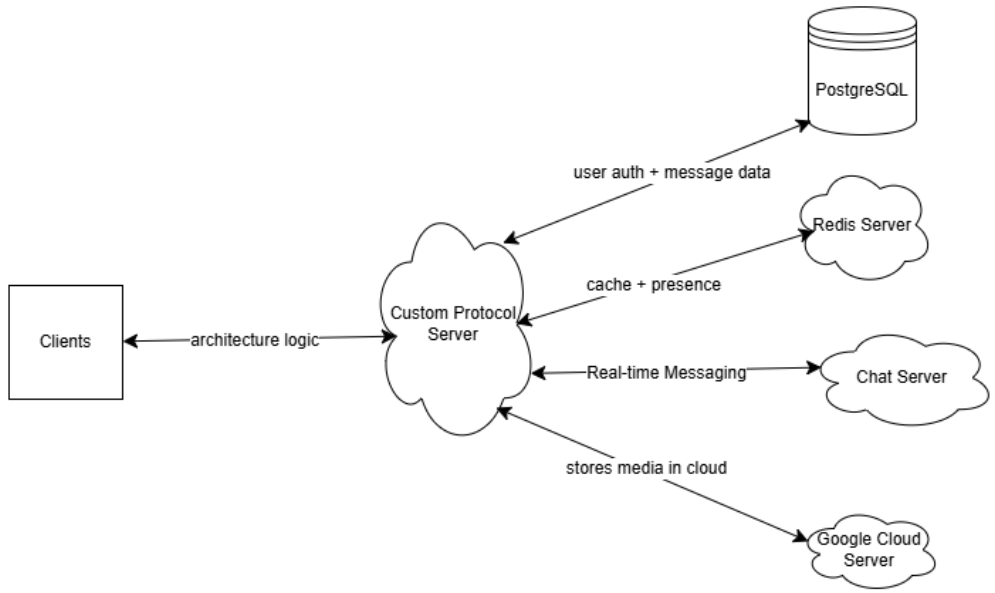

*Figure **1:**1: System Architecture Design*

Diagram Description:

The System Architecture Design above is made up of the following components:

- **Clients:**
  - Represents any end system that uses our chat application
  - This can be for text, call, or media transfer purposes
- **Custom Protocol Server**:
  - Handles client authentication
  - Manages database access and updating
  - Manages media upload to cloud storage
  - Establishes connection between clients
  - Signallling
  - Real time messaging
- **PostgreSQL:**
  - Used for user authentication
  - Used for data storage(chat history, chat records, call history)
  - Chat lookup

    - Media metadata and pointer storage
- **Redis Server:**
    - Used for in-memory storage(temporary storage). Plays the role of a cache.
    - Used to check client's presence(online status)
    - Used for message queuing(to ensure messages are received in the manner they were sent in).
- **Chat Server:**
    - Used for real-time messages.
    - Used for signalling to users when a peer-to-peer connection needs to be made
- **Google Cloud Storage:**
    - Used to store/download large media(audio,video, image) files.

The system has three different clients: text clients, call clients, and media clients. One client can play all three roles.

If text client:

- The clients interact with the Custom Protocol Server to send text messages to one another using the client-server communication model
- These clients only use three services: the Chat Server, the database, and the Redis Server.

If media client:

- The clients interact with the Custom Protocol Server to send media(audio, video, images) to one anotherusing the client-server communication model.
- These clients use four services: the Chat Server, the database, the Redis Server and the Google Cloud Storage

If call client:

- The clients interact with the Custom Protocol Server to connect to one another and get signal s using the peer-to-peer communication model
- These clients use only three services: the Chat Server(for call records), the database and the Redis Server.

Message flow can be both ways, with request and responses indicated by the bidirectional arrows. All clients interact only with the Custom Protocol Server directly. The other services are accessed through the Custom Protocol Server.

2. Application-Level Protocol Specification

Custom Application Protocol Design

Protocol Overview

- **Name**:
  - All-Round Connect Protocol (ARCP)
- **Transport**:
  - WebSocket (WSS): real-time messaging + calls
  - HTTPS REST: uploads, auth, history
- **Serialization format**:
  - JSON (dev mode): for debugging
  - MessagePack (Prod mode): to reduce user latency.

1. Roles of Communicating Entities

a. **Clients**:
   - **Role**: any hardware device or software application that initiates communication by requesting services, data, or resources from a central server.

b. **Custom Protocol Server**:
   - **Role**: The central server.
   - Core responsibilities: user interface, network communication, session management, local processing, real-time updates

c. **Chat Server**:
   - **Role**: Real-time messaging. Processes chat requests.
   - Core responsibilities: manage chat delivery and signalling

d. PostgreSQL
   - **Role**: The system's permanent storage
   - Core responsibilities: Stores user account information, chat history and media metadata.

e. **Redis Server**:
   - **Role**: The system's temporary storage, act as web cache.
   - Core Responsibilities: Online user tracking, temporary message queueing, caching frequently accessed data.

f. **Google Cloud Server**:

- **Role**: The system cloud-based storage, it receives any large media files sent by the client via the custom protocol server.
- Core Responsibilities: used to store large media files (audio, video, images), provide scalable storage as the number f users grows

2. Message Categories

a. Command Messages:

- Manage session setup, termination, and chat membership
- **Commands**:
  - SESS: start a session
  - END: End a session
  - AUTH: user login authentication
  - JOIN: join chat
  - LEAVE: leave chat
  - CALL: Start call
  - HANGUP: End call

b. Control Messages

- Handle acknowledgements, errors, and coordination
- **TYPES**:
  - SEND: send a message
  - UPLOAD: upload binary media (audio, video, image)
  - DELIVERED: confirm receipt
  - FAILED: reject message
  - ERROR: failure description
  - SYNC: State sync
  - ALIVE: Keep alive

c. Data Messages:

- Carry chat content and media data
- **TYPES**:
  - TEXT: chat text characterised by just a sequence of alpha-numeric letters.
  - MEDIA: file metadata (audio, video, image)
  - CALL_SIGNAL: WebRTC signalling
  - STATUS: typing/online

3. Message Structure

- A message must consist of:

    - **Header**: containing metadata
    - **Body**: containing application data
- ***Header***:


```
{
    "version": "1.0",
    "conv_id": "uuid",
    "method": "POST",
    "type": "DATA",
    "subtype": "TEXT",
    "command": "UPDATE_PROFILE",
    "sender": "user123",
    "session": "session123",
    "seq": 102,
    "auth": "jwt-token"
}
```


- ***Body***:
    - Example TEXT message


```
{
"text": "hello world"
}
```


4. The valid sequence of messages:

- ***Text Message***:
    - Client -> Custom Protocol Server (CPS): SendMessageRequest
    - CPS -> Database (DB): QueryUser
    - DB -> CPS: UserExists

    - CPS -> DB: QueryConversation
    - DB -> CPS: ConversationID

    - CPS -> Redis: StoreMessage
    - CPS -> PostgreSQL: StoreMessage

    - Redis -> CPS: OnlineStatus

    - CPS -> ChatServer: DeliverMessage
    - ChatServer -> RecipientClient: NewMessage

    - CPS -> SenderClient: 200 OK
- ***Binary Media Upload***:
    - Client -> CPS: UploadMediaRequest(metadatam authToken)
    - CPS -> PostgreSQL: ValidateUser(authToken)
    - PostgreSQL -> CPS: AuthValid

    - CPS -> GoogleCloudServer: RequestUploadURL(filetype, size)
    - GoogleCloudServer -> CPS: SignedUploadURL

    - CPS -> Client: UploadURLResponse(SignedURL)

    - Client -> GoogleCloudServer: UploadBinaryFile(SignedURL, fileData)
    - GoogleCloudServer -> Client: UploadSucess

    - GoogleCloudServer -> CPS: UploadComplete(fileID, metadata)

    - CPS -> PostgreSQL: StoreMediaReference(fileID, senderID, recipientID)
    - PostgreSQL -> CPS: Stored

    - CPS -> RedisServer: CacheMediaMetadata(fileID)
    - RedisServer -> CPS: Cached

    - CPS -> ChatServer: MediaMessageNotification(fileID, senderID, recipientID)
    - ChatServer -> RecipientClient(s): NewMediaMessage

    - CPS -> Client: 200 OK
- ***Peer-to-peer Call:***
    - CallerClient -> CPS: CallRequest(recipientID)

    - CPS -> PostgreSQL: CheckRecipientExists(recipientID)

- PostgreSQL -> CPS: Exists

- CPS -> RedisServer: CheckPresence(recipientID)
- RedisServer -> CPS: OnlineStatus

- CPS -> ChatServer: IncomingCallSignal(recipientID, callerInfo)
- ChatServer -> RecipientClient: IncomingCall

- RecipientClient -> CPS: CallAccept

- CPS -> CallerClient -> CallAccepted

- CallerClient -> CPS: WebRTCOffer(SDP)
- CPS -> RecipientClient: ForwardOffer(SDP)

- RecipientClient -> CPS: WebRTCAnswer(SDP)
- CPS -> CallerClient: ForwardAnswer(SDP)

- CallerClient -> CPS: ICECandidate
- CPS -> RecipientClient: ForwardCandidate

- RecipientClient -> CPS: ICECandidate
- CPS -> CallerClient: ForwardCandidate

- CallerClient ⇌ RecipientClient: P2P Media Stram (audio/video)

5. Expected Behavior in Normal and Error conditions:
   a. **Client's authentication**
      - Client sends HTTP GET request to Custom Protocol Server (CPS) to get the index page.
      - Custom Protocol Server responds with an index page that requires the user to authenticate.
      - The user sends an HTTP POST request to CPS, with message data being their username and password.
      - The CPS receives the request, reads the headers, and then extracts data from the request body.

- The server's application logic directs the request to the appropriate handler function or API endpoint based on the URL and HTTP method.
- The server-side code uses a library to establish a connection to PostgreSQL.
- The server-side code queries the database with extracted data.
- The database executes the query and returns the results back to the server.
- The server processes the data received from the database, generates an appropriate HTTP response including status code (200 OK or 404 Not Found) and the rendered HTML content.
- The response is sent back to the client over the network.
- The user's browser or application receives the response and updates the user interface, accordingly, redirecting to a new page or showing an error message.

b. Client's request to utilise application services:

- ***Text message utilisation:***
    - Client requests to SEND a text message to a specified user with a given user id.
    - The CPS receives the request, reads the headers, and then extract receiver's data from the request header.
    - The CPS's application logic directs the request to the appropriate handle function.
    - The server-side code queries the database with extracted data to check if recipient exists.
    - The database executes the query and returns the results back to the server.
    - The server processes the data received and generates an HTTP response with status code.
    - If status code is 200, the server then queries the database for the conversation ID to check if the conversation between the sender and the recipient exists.
    - The database then executes the query and returns the results back to the server.
    - If the conversation exists, the server inserts the new message from the request body to PostgreSQL and Redis server for permanent and in-memory storage.
    - The CPS sends a request to the Redis server to check if the recipients are online. If the recipient is online, the Redis server returns the response to the CPS.
    - The CPS sends the response with the updated chat history to the Chat Server for UI update. The message is broadcasted to recipient client(s).

    - If the recipient is offline, the Redis server stores the received text message(s) in the order they were received in.
    - The CPS sends an HTTP response with a 200 OK status back to the client.
    - If the status code is 404, the CPS returns a HTTP response to the client indicating that the login authentication was not successful.
- ***Media upload utilization:***
    - Client requests to UPLOAD a media file to a specified user with a given user id.
    - The CPS receives the request, reads the headers, and then extract receiver's data from the request header.
    - The CPS's application logic directs the request to the appropriate handle function.
    - The server-side code queries the database with extracted data to check if recipient exists.
    - The database executes the query and returns the results back to the server.
    - The server processes the data received and generates an HTTP response with status code.
    - If status code is 200, the server then queries the database for the conversation ID to check if the conversation between the sender and the recipient exists.
    - The database then executes the query and returns the results back to the server.
    - If the conversation exists, the server uploads the binary media file to Google Cloud Storage (GCS), returns the binary media metadata and file path to CPS.
    - CPS then stores the binary media metadata and file path to PostgreSQL and Redis server for permanent and in-memory storage.
    - The CPS then sends a request to the Redis server to check if the recipients are online. If the recipient is online, the Redis server returns the response to the CPS.
    - The CPS sends the response with the updated chat history to the Chat Server for UI update. The media metadata is broadcasted to recipient client(s).
    - If the recipient is offline, the Redis server stores the media metadata and other messages in the order they were received in.
    - The CPS sends an HTTP response with a 200 OK status back to the client.
    - If the status code is 404, the CPS returns a HTTP response to the client indicating that the login authentication was not successful.

- ***Call utilisation:***
    - Client requests to CALL a specified user with a given user id.
    - The CPS receives the request, reads the headers, and then extract receiver's data from the request header.
    - The CPS's application logic directs the request to the appropriate handle function.
    - The server-side code queries the database with extracted data to check if recipient exists.
    - The database executes the query and returns the results back to the server.
    - The server processes the data received and generates an HTTP response with status code.
    - If status code is 200, the server then queries the database for the conversation ID to check if the conversation between the sender and the recipient exists.
    - The database then executes the query and returns the results back to the server.
    - If the conversation exists, the server sends a request to the Redis Server to check if the recipients are online. If the recipient(s) are online, the Redis server returns the response to the CPS.
    - The CPS then stores the call metadata to PostgreSQL and Redis server for permanent and in-memory storage.
    - CPS then sends a signal to the Chat server to indicate that the call connection was successful.
    - The clients form a peer-to-peer connection.
    - The client the sends a request on HANGUP. The peer-to-peer connection is disabled.
    - The COS sends a signal to the Chat server to indicate that the call has ended.
    - The recorded duration of the call is then stored in the PostgreSQL and Redis Server as call metadata.
    - The CPS sends an HTTP response with a 200 OK status back to the client.
    - If the status code is 404, the CPS returns a HTTP response to the client indicating that the login authentication was not successful.

3. Design Rationale and Trade-Off Discussion

Design Rationale

System Architecture Design makes use of the following services for the following reasons:

1. ***Custom Protocol Server***
   - Acts as the central server.
   - Implements the client-server communication model, where all client interactions are done with the server ONLY.
   - Handles user authentication, manages database lookup and updates, manages media upload, establishes connection between users, used for signaling, and manages real-time messaging.
   - **BENEFIT**:
     - Allows unified control, making it easier to manage security, and back up data is a single location rather than on individual devices.
     - Multiple clients can simultaneously access shared resources, providing efficient resource sharing.
     - Improved data integrity because data is stored and managed centrally, and it is easier to ensure consistency and accuracy across all client applications.
   - **TRADE-OFF:**
     - If the server fails or becomes overloaded, all connected clients will lose access to all application services.
     - Too many simultaneous requests from clients can lead to server congestion, leading to slower performance.
     - Dependency of the entire system on the central server.
2. ***PostgreSQL Database (DB):***
   - Servers as primary storage.
   - Stores user authentication details, conversations, conversation participants, chat records, message history, and binary media's URL and metadata.
   - Used for lookups and updates to ensure consistency and accuracy.
   - **BENEFIT**:
     - Maintains coherent conversation flow across sessions as it ensures that chat history is never lost or corrupted due to sudden network failures or software crashes.
     - Ensures data integrity and prevents the occurrence of a message without valid senders.

    - Allows multiple users to read and write to the message database simultaneously which is essential for group chats.
  - **TRADE-OFF**:
    - Increased time and space complexity and performance overhead at extreme scales.
    - Replication lag might cause a user to send a message but not see it immediately if their app reads from a replica that hasn't synced yet.

3. ***Redis Server***
  - Plays the role of a cache.
  - Used to check the user's presence.
  - Used also for message queuing, where if a user is offline, the messages are queued in the order they were received in, pooled, and then sent to user when they are back online in the order they were received in.
  - **BENEFIT**:
    - Ultra-low latency: data is stored entirely in RAM, so messages can travel between users faster.
    - Manages volatile state data that changes rapidly, like online status.
    - Maintains a rolling "recent messages" buffer, ensuring no message is lost even if a client temporarily goes offline.
    - High concurrency allows if to manage thousands of simultaneous chat connections without memory overhead.
  - **TRADE-OFF:**
    - Prioritize speed over permanent safety, so there is a risk of data loss.
    - All active data must fit in RAM, which requires massive amounts of expensive RAM.

4. ***Google Cloud Server***
  - Used for cloud storage.
  - Stores binary media (audio, video, images) that clients send to one another.
  - **BENEFIT:**
    - Infinite Scalability: disk space is not affected by growing user base.
    - Data is replicated across multiple disks to ensure chat logs and media remain available even if the entire region faces an outage.
    - Extremely fast and cheap lookups.
    - Cloud storage is built-in to serve blobs (Binary Larger Objects)
  - **TRADE-OFF**:
    - If the server's connection to Google Cloud is interrupted, your application loses access to all stored assets.

    - Poor real-time performance compared to PostgreSQL.

5. ***Chat Server***
   - Executed application-level rules, send messages and signals to users when a call is made.

Our chat system uses hybrid architecture, combining both Client-Server and Peer-to-Peer communication.

The different communication models are being used depending on the type of data being sent (text, audio, image, and video media). This allows us to balance reliability, performance, and server load.

Client-Server Model

- Clients interact with the Custom Protocol Server to start sessions, establish connections, send text messages, and media to other clients.
- Clients do not communicate directly for data transfers; they do so through the Custom Protocol Server as speed is not the priority, but rather integrity.
- ***Reasons***:
  1. **User Membership Authentication**:
     - For clients to utilize the services of the chat application, they must first be users of the application.
     - Clients cannot authenticate each other's membership directly, so they can only achieve this through the server.
  2. **Stable and Reliable Connection**:
     - For clients to communicate, a connection must be established first. Through the server, the connection between clients is established reliably via the Transport Connection Protocol (TCP).
     - Once clients are connected, they can then access services provided by the Chat Application through the server.
  3. **Data integrity and completeness:**
     - When clients communicate through the server, the data is transferred from one end system process to another on the internet using Transport Connection Protocol (TCP).
     - TCP ensures that data gets to the right place intact, which is exactly what is needed for text and media data transfers.

Peer-To-Peer Model

For services like audio and video calls, clients can communicate directly; however, that should be after a connection is established. This is why the chat application uses hybrid architecture.

Peer-to-peer (P2P) communication is integrated for the following:

1. **Reduce server load**
- Audio and video calls involve real time streaming when they occur; this data can consume a lot of bandwidth should it be transmitted through the server. P2P allows users to stream directly to each other once a connection is established.
2. **Faster data transfers**
    - Direct client-client communication can be faster as data transfer is not directly through the server.

4. Use of AI Declaration.

AI (Chat GPT) was used for the following:

- To find servers like (Redis) which can provide features like data cache, message queuing, and presence detection.
- To explicitly explain and understand the flow of data during a peer-to-peer connection.
- To design the valid sequence of messages.
- Used to get a template of designing your own application protocol.
- To understand the difference between using only Redis and a primary database.
- To decide which primary database to use (PostgreSQL)
- To understand the importance of using Cloud storage for storing the binary media and selecting the choice of Cloud server.